

딥러닝 시스템 설계 · 7주차

# Transformer를 이해하는 하나의 긴 강의노트

김지훈 교수

2026. 07. 31.

## 문장을 계산 가능한 단위로 바꾸기

언어 모델의 첫 단계는 문장을 토큰의 열로 바꾸는 토큰화다. 토큰은 반드시 단어와 일치하지 않는다. 자주 등장하는 단어는 하나의 토큰이 되기도 하고, 드문 단어는 여러 조각으로 나뉘기도 한다. 이 선택은 모델이 보게 되는 시퀀스의 길이와 어휘 사전의 크기를 동시에 결정한다.

BPE는 처음에 문자처럼 작은 단위에서 시작해 데이터에서 자주 붙어 등장하는 쌍을 반복적으로 합친다. 고정된 거대 단어 사전을 외우는 대신, 익숙한 단어는 짧게 표현하고 새로운 단어는 이미 아는 조각의 조합으로 처리한다. 토큰 수가 줄면 뒤의 어텐션 연산량도 함께 줄어들 수 있다.

토큰 ID 자체에는 의미 있는 거리가 없으므로 임베딩 테이블을 통해 연속 벡터로 바꾼다. 학습 과정에서 비슷한 문맥에 등장하는 토큰의 벡터는 비슷한 방향을 갖게 된다. 이후의 모든 블록은 이 벡터 열을 입력으로 받아 문맥을 반영한 표현으로 갱신한다.

## 순서가 사라진 연산에 위치를 되돌려 주기

어텐션만 놓고 보면 입력 순서를 바꾸어도 동일한 방식으로 계산된다. 하지만 언어에서는 “개가 사람을 물었다”와 “사람이 개를 물었다”가 다르다. 위치 인코딩은 각 토큰 표현에 순서 정보를 더해 모델이 같은 단어라도 등장 위치에 따라 다르게 해석하도록 만든다.

RoPE는 위치에 따라 Query와 Key 벡터를 서로 다른 각도로 회전시킨다. 두 벡터의 내적에는 절대 위치보다 상대적 거리가 자연스럽게 반영된다. 학습 때 본 길이를 넘어서는 문맥에서 얼마나 안정적인지는 회전 주파수와 스케일링 방법에 영향을 받는다.

## Query, Key, Value와 문맥의 재구성

각 토큰 표현은 서로 다른 선형 변환을 지나 Query, Key, Value가 된다. Query는 지금 찾고 싶은 정보의 조건, Key는 자신이 어떤 정보인지 알리는 표지, Value는 실제로 전달할 내용으로 볼 수 있다. Query와 Key의 유사도가 높을수록 해당 Value가 결과에 더 크게 섞인다.

Scaled Dot-Product Attention은 Query와 모든 Key의 내적을 구한 뒤 차원의 제곱근으로 나눈다. 차원이 커질수록 내적의 분산이 커져 Softmax가 지나치게 뽕족해지는 현상을 막기 위한 조정이다. 그 점수에 Softmax를 적용하고 Value의 가중합을 만들면 한 토큰의 새 문맥 표현이 된다.

Softmax는 여러 점수를 합이 1인 양의 가중치로 바꾼다. 가장 큰 점수를 가진 위치가 상대적으로 강조되지만 나머지 위치의 정보도 완전히 사라지지는 않는다. 온도가 낮아지면 분포가 더 날카로워지고, 높아지면 여러 위치를 고르게 참고한다.

#### 04 · SELF-ATTENTION

## 한 문장 안에서 서로를 바라보는 방식

Self-Attention에서는 Q, K, V가 모두 같은 입력 시퀀스에서 나온다. 한 위치는 자신의 Query로 다른 모든 위치의 Key를 조회하고, 필요한 Value를 모아 문맥화된 새 표현을 만든다. 따라서 멀리 떨어진 단어 사이의 관계도 한 번의 층에서 직접 연결할 수 있다.

Multi-Head Attention은 표현 차원을 여러 머리로 나눠 서로 다른 투영 공간에서 어텐션을 병렬 계산한다. 어떤 머리는 문법적 의존성을, 다른 머리는 지시 대상이나 위치 관계를 더 강하게 포착할 수 있다. 각 머리의 출력을 이어 붙인 뒤 다시 투영해 다음 연산으로 보낸다.

Grouped-Query Attention은 여러 Query 머리가 더 적은 수의 Key/Value 머리를 공유하게 한다. Multi-Head Attention의 표현력을 상당 부분 유지하면서도 생성 과정에서 보관해야 하는 Key/Value의 양을 줄이는 절충안이다. 공유 범위를 넓힐수록 메모리는 줄지만 머리별 독립성도 낮아진다.

#### 05 · CAUSAL MASKING

## 미래를 보지 않고 다음 토큰을 예측하기

생성형 언어 모델은 현재 위치에서 미래 정답을 미리 보면 안 된다. Causal Mask는 현재 위치보다 오른쪽에 있는 점수를 음의 무한대로 바꾼 뒤 Softmax를 적용한다. 그 결과 각 토큰은 자기 자신과 앞쪽 토큰에만 주의를 줄 수 있다.

이 마스킹은 단순히 답을 가리는 장치가 아니다. 새 토큰이 한 개 추가되어도 이미 계산된 앞쪽 토큰들은 새 토큰을 볼 수 없으므로 그 위치들의 표현은 바뀌지 않는다. 뒤에서 추론 최적화를

생각할 때 이 불변성이 매우 중요한 출발점이 된다.

실제 생성은 지금까지의 토큰으로 다음 토큰의 확률 분포를 만들고, 하나를 선택해 입력 끝에 붙이는 일을 반복한다. 이 자기회귀 과정은 스텝 사이에 의존성이 있어 토큰 방향으로 완전히 병렬화할 수 없다. Greedy Decoding은 매번 확률이 가장 큰 토큰 하나만 고르는 가장 단순한 선택 규칙이다.

대화에서 추가 · 추론 최적화

## KV Cache: 바뀌지 않는 Key와 Value 재사용하기

자기회귀 추론의 매 스텝에서 과거 토큰의 Key와 Value는 이미 계산되어 있고 Causal Mask 때문에 새 토큰의 영향을 받지 않는다. KV Cache는 이 텐서들을 레이어별로 저장한 뒤 다음 스텝에서 다시 사용하고, 새 토큰 한 개의 K/V만 뒤에 추가한다.

연산량은 크게 줄지만 캐시 크기는 배치, 레이어 수, KV 헤드 수, 문맥 길이와 head dimension에 비례해 증가한다. 긴 문맥 서비스에서는 모델 가중치보다 캐시 메모리가 더 큰 병목이 될 수 있으므로 GQA나 페이지 단위 메모리 관리와 함께 고려한다.

대화에서 추가 · 생성 루프

## Autoregressive Decoding: 한 토큰씩 이어지는 실행 상태

강의노트는 자기회귀 생성을 토큰 선택 규칙의 배경으로만 짧게 언급했다. 대화에서는 이를 별도 실행 상태로 분리했다. 현재까지의 토큰, 종료 조건, 샘플링 설정과 캐시 위치가 한 스텝에서 다음 스텝으로 함께 전달된다.

토큰 방향 의존성 때문에 한 요청 안의 생성 스텝은 순차적이지만, 서버는 서로 다른 요청을 배치로 묶어 가속기를 채울 수 있다. 따라서 모델 알고리즘의 순차성과 서비스 전체 처리량은 구분해 이해해야 한다.

06 · TRANSFORMER BLOCK

## 어텐션을 깊은 네트워크로 쌓는 구조

Transformer Block은 어텐션과 위치별 Feed-Forward Network를 중심으로 구성된다. 어텐션이 토큰 사이에서 정보를 섞는다면, Feed-Forward Network는 각 위치에서 같은 비선형 변환을 독립적으로 수행해 특징을 확장하고 압축한다. 이 블록을 여러 층 쌓으면 더 추상적인 문맥 표현을 얻는다.

Residual Connection은 하위 층의 입력을 변환 결과에 더해 깊은 네트워크에서도 기울기가 안정적으로 흐르게 한다. 모든 층이 표현을 처음부터 다시 만드는 대신 기존 표현에 필요한 변화량을 보태도록 학습할 수 있다.

Layer Normalization은 한 토큰 벡터 안의 특성들을 정규화한다. 배치 크기나 시퀀스 길이에 덜 의존하므로 가변 길이 언어 입력에 잘 맞는다. 정규화를 잔차 연결의 앞에 돌지 뒤에 돌지는 깊은 모델의 학습 안정성에 큰 영향을 준다.

### 07 · MODEL FAMILIES

## Encoder와 Decoder가 맡는 역할

Encoder는 입력 전체를 양방향으로 읽어 문맥 표현을 만들고, Decoder는 지금까지 생성한 출력만 보며 다음 토큰을 만든다. 번역 모델에서는 Decoder가 Cross-Attention을 통해 Encoder의 표현을 조회한다. 반면 GPT 계열은 Decoder 블록만 쌓아 다음 토큰 예측에 집중한다.

Cross-Attention에서는 Query가 현재 Decoder 상태에서 나오고 Key와 Value는 Encoder 출력에서 나온다. Self-Attention과 계산식은 같지만 정보의 출처가 다르다. 이 구분은 멀티모달 모델에서 텍스트가 이미지 특징을 조회하는 구조로도 확장된다.

### 08 · TRAINING AND INFERENCE

## 학습의 병렬성과 추론의 순차성

학습에서는 정답 문장 전체가 이미 주어진다. 입력을 한 칸 밀어 각 위치의 다음 토큰 정답을 만들고, Causal Mask를 적용한 한 번의 Forward Pass로 모든 위치의 손실을 동시에 계산한다. 이후 역전파로 모든 층의 파라미터를 갱신한다.

Teacher Forcing을 사용하는 학습에는 토큰을 하나 생성한 뒤 그 결과를 기다리는 “이전 생성 스텝”이 없다. 같은 시퀀스의 모든 위치를 행렬 연산으로 병렬 처리하기 때문에, 순차 추론과 학습의 병목은 서로 다른 곳에서 생긴다.

추론에서는 아직 정답이 없으므로 방금 선택한 토큰이 다음 스텝의 입력이 된다. 모델 가중치는 고정되어 있지만 문맥 길이는 매 스텝 늘어난다. 처리량, 첫 토큰 지연시간, 토큰 간 지연시간을 구분해야 서비스 병목을 정확히 설명할 수 있다.

## 09 · DECODING STRATEGIES

# 확률 분포에서 실제 문장을 선택하기

Greedy Decoding은 각 스텝에서 가장 높은 확률의 토큰을 고른다. 빠르고 결정적이지만 초반의 작은 선택 실수를 되돌릴 수 없고, 전체 문장의 확률이 가장 높은 경로를 보장하지 않는다.

Beam Search는 매 스텝에서 상위 후보 경로를 여러 개 유지한다. 번역처럼 정답성이 중요한 작업에서는 유용하지만 후보 수만큼 계산과 메모리가 늘고, 열린 대화에서는 지나치게 평범한 문장을 만들 수 있다.

Temperature Sampling은 로짓을 온도로 나눈 뒤 확률적으로 토큰을 뽑는다. 낮은 온도는 확실한 후보에 집중시키고 높은 온도는 드문 후보의 가능성을 키운다. Top-k나 Top-p와 함께 사용하면 다양성과 안정성을 조절할 수 있다.

## 10 · SYSTEMS PERSPECTIVE

# 같은 수식도 메모리 이동에 따라 속도가 달라진다

가속기에서 실제 속도는 부동소수점 연산 횟수만으로 결정되지 않는다. 큰 텐서를 HBM에서 읽고 다시 쓰는 비용이 연산보다 더 비쌀 수 있다. 따라서 동일한 결과를 내더라도 중간 행렬을 어떻게 나누고 언제 메모리에 기록하는지에 따라 실행 시간이 달라진다.

긴 문맥에서는 어텐션 점수 행렬의 크기가 시퀀스 길이의 제곱으로 증가한다. 정확한 결과를 유지하면서 작은 타일 단위로 계산하고 중간값을 재사용하면 메모리 왕복을 줄일 수 있다. 알고리즘과 시스템 구현을 함께 봐야 하는 대표적인 지점이다.

이 강의노트의 개념들은 독립된 목록이 아니다. 토큰화가 시퀀스 길이를 만들고, 위치와 어텐션이 문맥을 구성하며, 마스킹이 생성의 방향을 정하고, 블록 구조와 디코딩 전략이 실제 모델 동작으로 이어진다. 그래프는 이 의존 관계를 한눈에 보기 위한 또 하나의 읽기 방식이다.

대화에서 추가 · IO 최적화

## Flash Attention: 중간 행렬을 쓰지 않는 정확한 어텐션

Flash Attention은 근사 어텐션이 아니다. Query, Key, Value를 작은 타일로 나누고 온라인 Softmax를 사용해 전체 점수 행렬을 HBM에 기록하지 않으면서도 일반 어텐션과 같은 결과를 계산한다.

KV Cache가 생성 스텝 사이의 상태를 보존하는 최적화라면 Flash Attention은 한 번의 어텐션 연산 안에서 메모리 이동을 줄이는 최적화다. 둘은 함께 사용할 수 있지만 해결하는 병목의 축이 다르다.